

POGRAMMATION ORIENTÉE OBJET EN JAVA

Prof. Amina ADADI
a.adadi@umi.ac.ma

Année universitaire 2024-2025

Objectifs du cours

Ce cours fournit une bonne expérience théorique et pratique de la programmation orientée objet (P.O.O) avec java, il s'articule autour de trois principaux objectifs :



Maitriser les concepts basiques du langage java (structures de données, structure de contrôles, objets prédéfinis).



Comprendre le paradigme OO et utiliser le langage java pour le mettre en œuvre.



Manipuler les APIs Java: interfaces graphiques, prog évènementielle, gestion des BD, prog concurrentielle...

- **Partie 1: langage java**
 - Chapitre 1: Langage de programmation Java
- **Partie 2: Programmation orientée objet**
 - Chapitre 2: Classes et objets
 - Chapitre 3: Héritage
 - Chapitre 4: Collections
- **Partie 3: APIs java**
 - Chapitre 5: Programmation graphique et évènementielle
 - Chapitre 6: Manipulation des bases de données en java

Déroulement du cours

Volume horaire total		12 semaines			
Cours et TD	2h/semaine	TP	2h/semaine		
Evaluation					
Devoirs surveillés	2	Colle TP	1	Devoirs	A définir (travaux individuels ou par groupe, réalisés pendant ou en dehors des séances de cours/TD/TP)
+ NOTE de présence, assiduité, et participation					

- Pour chaque section et pour chaque groupe de TP désigner un représentant, ce représentant sera en charge de vous communiquer les informations urgentes et imprévues en temps réel et de m'informer, en nom de la classe, de toute information nécessaire pour le bon déroulement du cours.
- Une plateforme (Classroom) sera mise à votre disposition pour le reste des échanges (communication de cours, remise des devoirs...), je vous conseille donc de la consulter en régulier.

Partie 1: Langage JAVA

Prof. Amina ADADI
a.adadi@umi.ac.ma

Année universitaire 2024-2025

Chapitre 1: Langage de programmation Java

Points à discuter:

- Syntaxe de base
- Types de données primitifs
- Opérateurs
- Structures de contrôle
- Références d'objet
- Chaines de caractères et tableaux



Langage JAVA

Le langage Java:

- est né en 1995 chez Sun Microsystems (racheté par Oracle)
 - Version actuelle Java 23
- est **orienté objet**
- est **fortement typé**
 - Toute variable doit être déclarée avec un type
 - Le compilateur vérifie que les utilisations des variables sont compatibles avec leur type
 - Les types sont d'une part fournis par le langage, mais également par la définition des classes
- est **compilé**
 - En bytecode, i.e., code intermédiaire indépendant de la machine
- est **interprété**
 - Le bytecode est interprété par une machine virtuelle Java



Les principales raisons du succès de Java

1. C'est un langage **orienté objet** dérivé du C, mais plus simple à utiliser et plus «pur» que le C++. On entend par « pur » le fait qu'en Java, on ne peut faire que de la programmation orientée objet contrairement au C++ qui reste un langage hybride, c'est-à-dire autorisant plusieurs styles de programmation. C++ est hybride pour assurer une compatibilité avec le C ;
1. Il est **multi plates-formes** : les programmes tournent sans modification sur tous les environnements où Java existe (Windows, Unix et Mac) ;
2. Il est doté, en standard, de bibliothèques de classes très riches comprenant la gestion des interfaces graphiques (fenêtres, boîtes de dialogue, contrôles, menus, graphisme), la programmation concurrente (multitâches), la gestion des exceptions, les accès aux fichiers et au réseau ... L'utilisation de ces bibliothèques facilitent grandement la tâche du programmeur lors de **la construction d'applications complexes de type client lourd, web ou mobile** ;

Premier code java

Dans un fichier de nom HelloWorld.java

```
public class HelloWorld{  
    public static void main(String[] args) {  
        System.out.print("Bonjour tout Le monde !");  
    }  
}
```



Ça définit une classe, qui est une unité de compilation



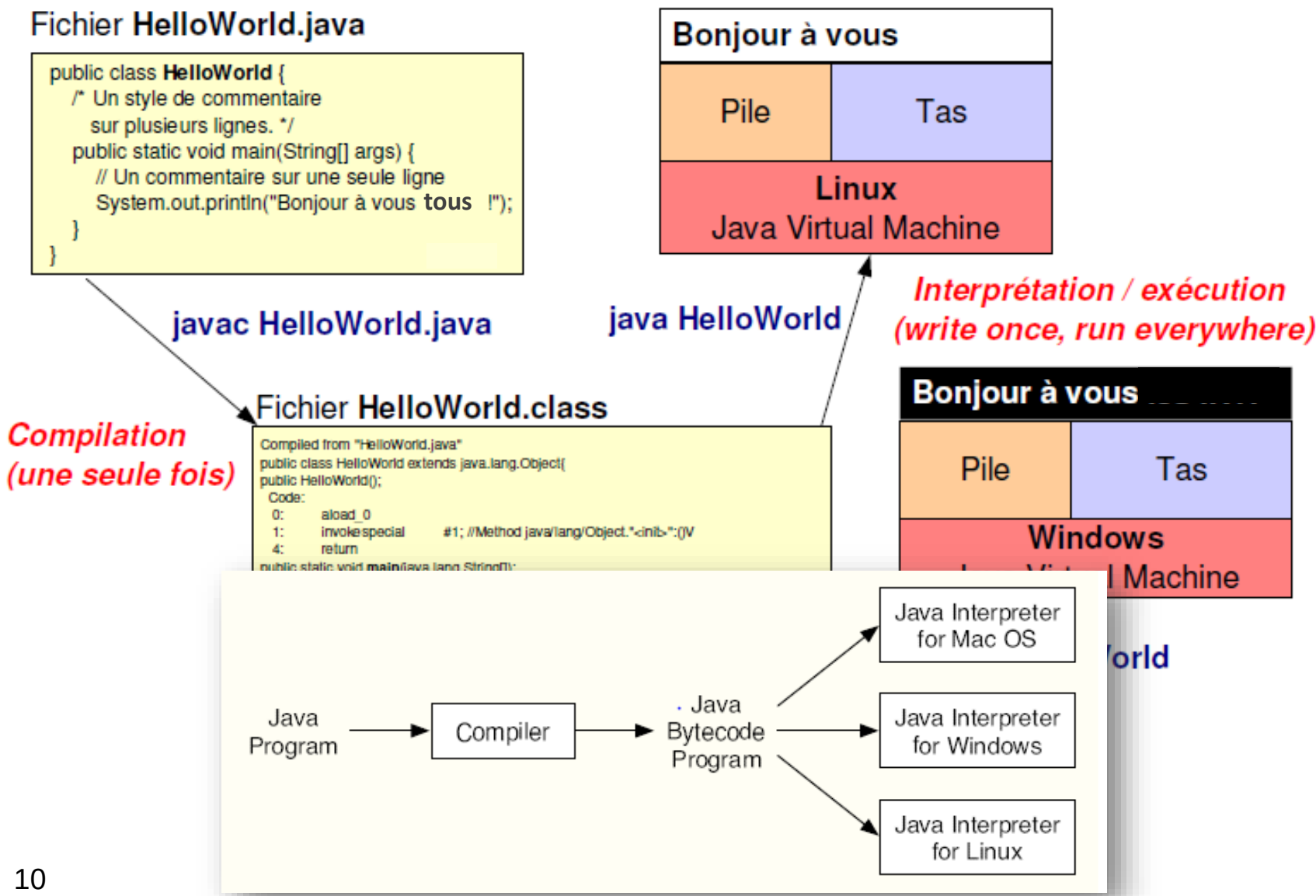
Comme il y a une méthode `main`, cette classe est « *exécutable* »



- **Règle 1: toute classe publique doit être dans un fichier qui a le même nom que la classe**
- **Règle 2: tout code doit être à l'intérieur d'une classe**

Mécanismes de fonctionnement de Java

- Compilation du langage source -> exécution du bytecode



Le bytecode

- Le **langage source Java** est défini par la JLS (*Java Language Specification*) éditée par Sun-Oracle
 - Dans sa syntaxe et sa sémantique
- Le code source d'une classe contenue dans un fichier est compilé avec la commande **javac**
 - Cela produit un code intermédiaire, appelé bytecode, qui est le « langage machine » de la machine virtuelle Java
- Le **bytecode** d'une classe est destiné à être **chargé** par une **machine virtuelle** qui doit l'exécuter avec la commande **java**
 - Soit par **interprétation**, soit par **compilation** « juste à temps » (*just-in-time* ou JIT)
 - L'argument est le nom de la classe (sans l'extension `.class`)

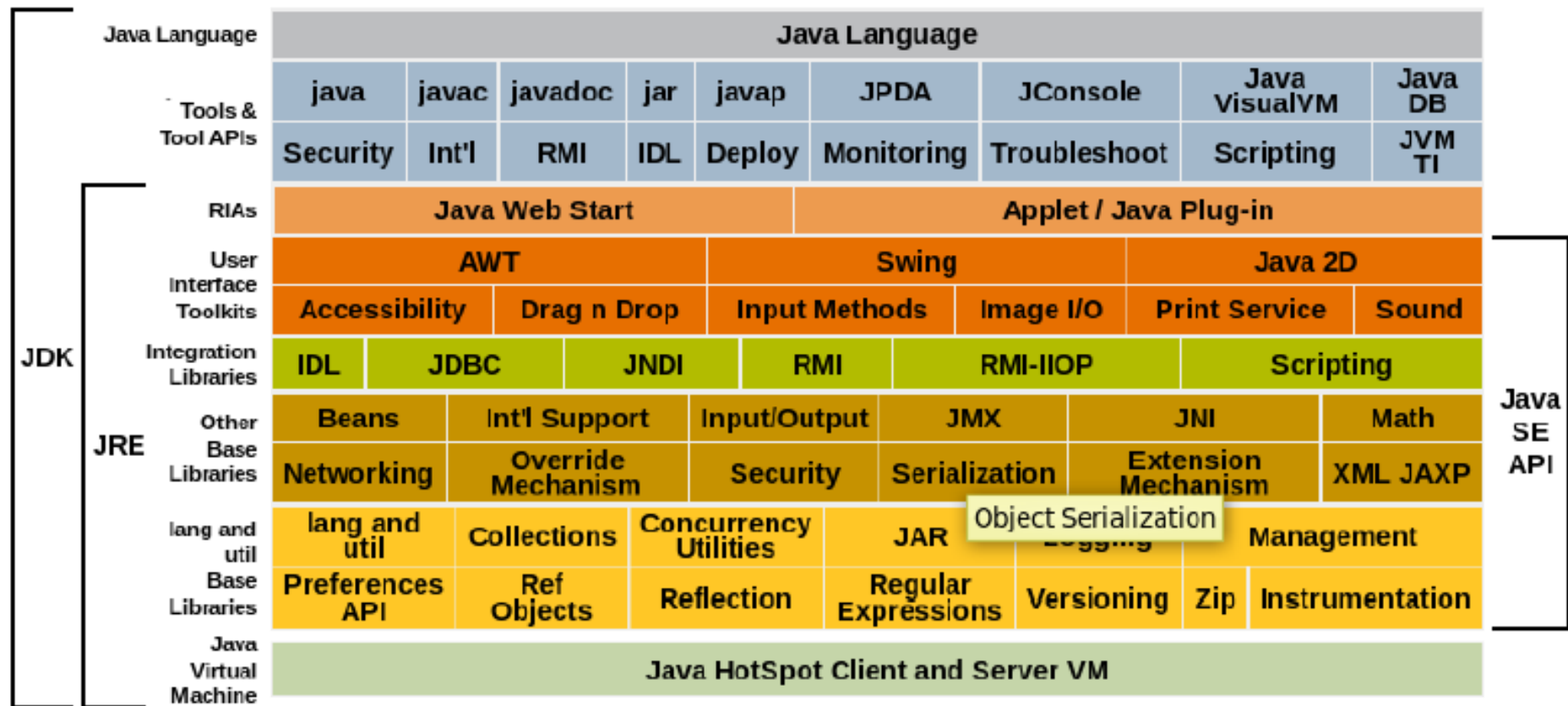
La machine virtuelle (JVM)

- Son rôle est d'**abstraire le comportement d'une machine**
 - Pour le rendre le + possible indépendant de la plateforme
 - Son comportement est défini par la JVM Spec édité par Sun-Oracle
- Une JVM est une implémentation de cette spec
 - Qui peut être adaptée à une plateforme d'accueil (Windows, Linux, Mac...)
 - Qui peut être développée par Sun (HotSpot: open source GPL depuis 2006) ou par d'autres: IBM, Jikes, etc.
- Une JVM **traduit** le **bytecode** dans le **langage machine** de la plateforme d'accueil

Java: un langage et une plateforme

- Dans la technologie Java, on a donc besoin
 - Du **langage de programmation** et du compilateur
 - Et plein de commandes bien utiles: jar, javap, javadoc, etc
 - De la **JVM** et des **APIs** (*Application Programming Interfaces*) regroupées dans une « plateforme »:
 - Java SE (Java Platform, Standard Edition): Java SE 6 pour applications classiques, desktop
 - Java EE (Java Platform, Enterprise Edition): Java EE 6 pour développer et déployer des applications serveur, Web services, etc.
 - Java ME (Java Platform, Micro Edition): J2ME pour les applications embarquées, PDA, téléphones, etc.
 - Si on veut juste exécuter, il suffit du **JRE** (*Java Runtime Execution*) par opposition au **JDK** (*Java Développement Kit*)

Mécanismes de fonctionnement de Java



Documentation :<https://docs.oracle.com/javase/10/docs/api/>

Exercice d'application 1

Reprendre le programme HelloWorld.java, le saisir avec l'éditeur d'eclipse et l'exécuter.

Décompiler le fichier HelloWorld.class en utilisant la commande javap (avec option -c).

Questions

1. Est ce que le code suivant est correcte:

```
int a,b;  
Pulic calss somme {  
public static void main {  
System.out.println("la somme est"+ a+b);  
} }
```

2. Est-ce que j'ai besoin de mettre un main dans toutes les classes que je code?

3. Java est un langage compilé ou interprété?

4. Quel composant est utilisé pour compiler, déboguer et exécuter un programme java?

5. Quel composant est responsable de la conversion du byte-codes en code spécifique à la machine?

6. Quel composant est responsable de l'exécution du programme Java?

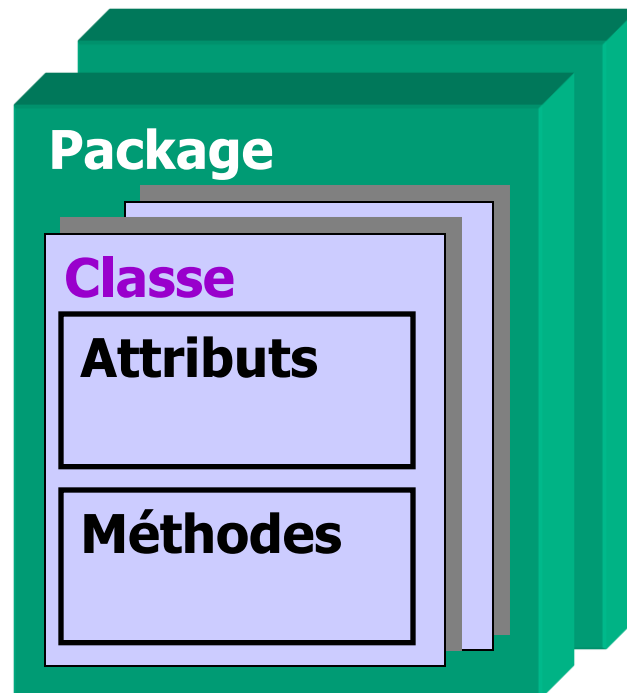
7. Est-ce qu'un programme Java, codé sous Windows, peut être compilé sous Mac et enfin exécuté sous Linux?

8. Quelle est l'extension d'un programme Java?

9. Quelle est l'extension d'un programme compilé?

Syntaxe : généralités

- Une application Java utilise un ensemble de classes
- Les classes sont regroupées par package
- Une classe regroupe un ensemble d'attributs et de méthodes
- Une application possède un point d'entrée fourni par la méthode « main » (sauf les applets)



Syntaxe : généralités

- Le nom de la classe est spécifié derrière le mot clé « class »
- Le corps de la classe est délimité par des accolades
- On définit dans le corps les attributs et les méthodes qui constituent la classe

```
class Test {  
  
    public static void main (String[] args)) {  
        ...  
        // corps de la méthode main  
    }  
  
}
```

Syntaxe : généralités

Comme dans de nombreux langages informatiques, un code source Java est composé de “mots” qui sont délimités par des caractères (ou des ensembles de caractères) particuliers. Ces “mots” peuvent être :

- des identificateurs : `nomDeVariable`, ...
- des mots-clefs réservés : `if`, `for`, `private`...
- des littéraux : `3.14`, `“une chaîne”`, ...
- séparateurs : blancs, commentaires
- séparateurs de structure : `{`, `[`, `;`, ...
- opérateurs : `+`, `*`, `&&`, `>>=`, ...
- Un commentaire:

```
if (variable < 0) { variable=0; // un commentaire est-il nécessaire?  
    }
```

Commentaires

Trois notations sont possibles pour l'insertion de commentaires:

// commentaire sur une ligne

/* commentaires sur une ou plusieurs lignes */

/** insertion de documentation */

Les commentaires pour la documentation sont placés juste avant une déclaration JAVA et indiquent que son contenu doit être inclus automatiquement dans la documentation générée par l'outil *javadoc*.

Séparateurs

Une instruction se termine par un point virgule (;).

Elle peut contenir une ou plusieurs lignes.

```
total = ix + iy + iz + ij + ik + il ; // une instruction
```

est la même chose que :

```
total = ix + iy + iz  
      +  ij + ik + il ; // deux lignes, une instruction
```

Séparateurs

Un bloc regroupe un ensemble d'instructions, il est délimité par des accolades.

```
{ // un bloc ix = iy + 1 ; iz = 25 ;  
}
```

Les blocs peuvent s'emboîter les uns dans les autres :

```
    { int ix = iy + 1 ; while ( ik < MIN ) { // bloc emboîté  
ik = ik * ix;  
    }  
}
```

Séparateurs

Les espacements (caractère espace, tabulation, retour chariot) peuvent se répéter à l'intérieur du code source. On peut les utiliser pour améliorer la présentation.



```
if (ix<iy) ix=iy; //compact  
  
if ( ix < iy )  
{  
    ix = iy ;  
} // plus aéré
```

Identificateurs

- Dans le langage Java un identificateur commence par une lettre, un trait de soulignement (_), ou un signe dollar (\$). Les caractères suivants peuvent également comporter des chiffres.
- Les identificateurs opèrent une distinction entre majuscules et minuscules et n'ont pas de limitation de longueur.
- Identificateurs valides:
 - ✓ identificateur
 - ✓ fenetre
 - ✓ nomUtilisateur
 - ✓ MaClasse
 - ✓ _var_sys
 - ✓ \$picsou
- Les identificateurs peuvent contenir des mots-clés, mais ne doivent pas eux même être des mots-clés: "longueur" est un identificateur valide mais "long" ne l'est pas.

Mots-clés

<code>abstract</code>	<code>do</code>	<code>implements</code>	<code>private</code>	<code>this</code>
<code>boolean</code>	<code>double</code>	<code>import</code>	<code>protected</code>	<code>throw</code>
<code>break</code>	<code>else</code>	<code>instanceof</code>	<code>public</code>	<code>throws</code>
<code>byte</code>	<code>extends</code>	<code>int</code>	<code>return</code>	<code>transient</code>
<code>case</code>	<code>false</code>	<code>interface</code>	<code>short</code>	<code>true</code>
<code>catch</code>	<code>final</code>	<code>long</code>	<code>static</code>	<code>try</code>
<code>char</code>	<code>finally</code>	<code>native</code>	<code>strictfp</code>	<code>void</code>
<code>class</code>	<code>float</code>	<code>new</code>	<code>super</code>	<code>volatile</code>
<code>continue</code>	<code>for</code>	<code>null</code>	<code>switch</code>	<code>while</code>
<code>default</code>	<code>if</code>	<code>package</code>	<code>synchronized</code>	<code>assert (1.4)</code>

Variables

- Une variable possède un type et un nom
- Le type peut être **un type primitif ou référence d'objet** (Strings, Objects, Enums..)
- L'initialisation d'une variable peut se faire au moment de la déclaration
- Une variable possède **une portée** qui s'étend jusqu'à la fin du bloc dans lequel elle est définie

```
{  
    int compteur;  
    int indice = 0;  
  
    Voiture golf;  
    Voiture toyota = new Voiture();  
}
```

Variables: *Portée d'une variable*

- Le langage Java définit précisément la “portée” d’une variable : c’est à dire la partie de code dans laquelle un symbole déclaré (le “nom” d’une variable) peut être utilisé et a un sens.
- Dans un bloc de code on ne “voit” (on ne peut utiliser) que les symboles définis dans le bloc courant , ou le bloc englobant.

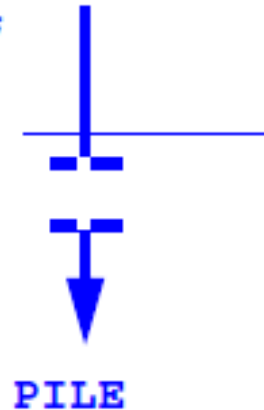
```
// bloc englobant
int nombre ;
.....
{ // bloc interne int
    autreNombre ;
    ....
    nombre = 10 ; // correct
} // fin bloc interne ... autreNombre = 7
; // ERREUR ! IMPOSSIBLE nombre = 20 ; //
possible
}
```

Variables: *Types primitifs et références d'objets*

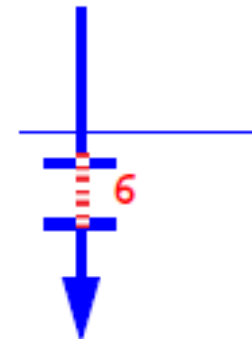
Le langage Java distingue les types scalaires primitifs des autres types (objets).

- Dans un bloc de code la déclaration d'un type scalaire primitif, comme *boolean*, *int*, *float*, alloue un emplacement pour stocker une valeur du type considéré.

```
{//bloc de code exécutable -variables automatiques-  
  int valeur ;
```

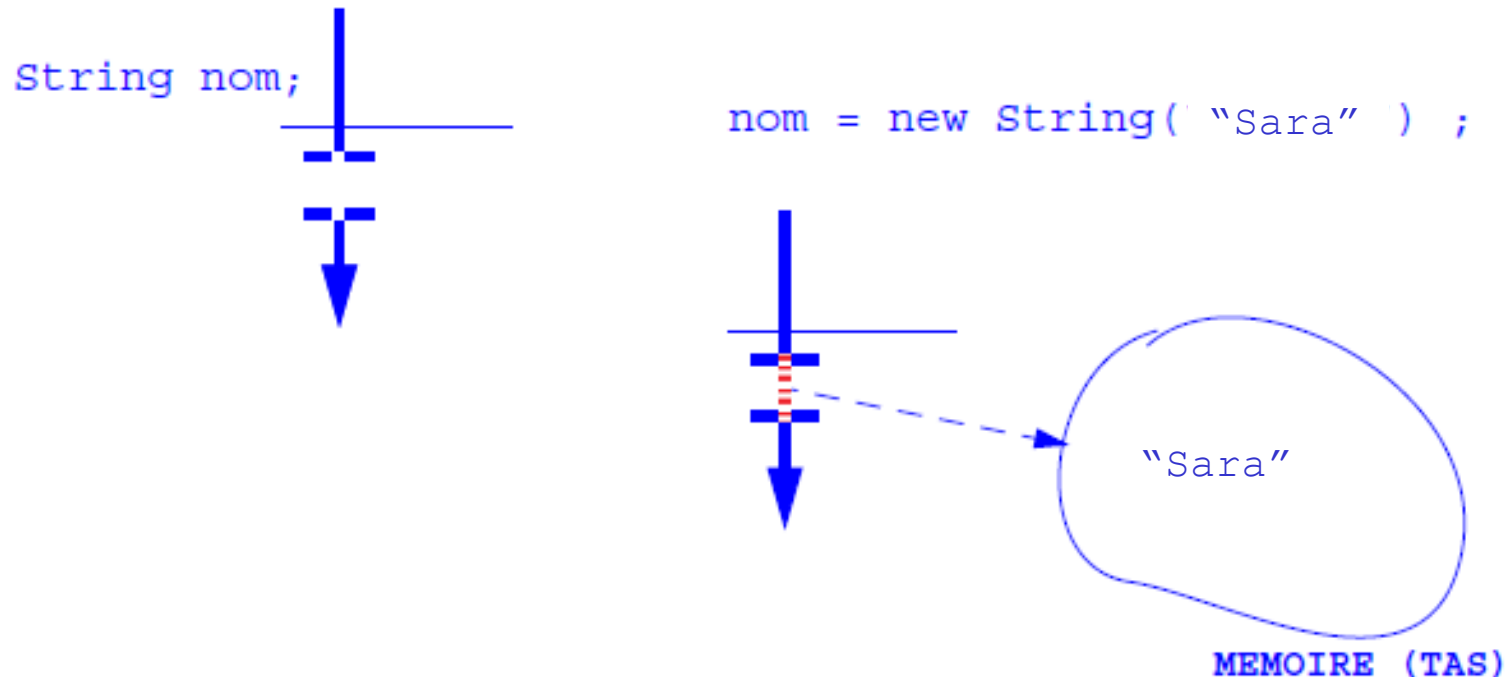


```
    valeur = 6 ;
```



Types primitifs et références d'objets

- La déclaration d'une variable "objet" ne crée pas d'emplacement en mémoire pour stocker l'objet mais crée un emplacement pour stocker une référence vers un objet.



Types primitifs : logiques

Les valeurs logiques ont deux états : vrai ou faux. En Java une telle valeur est représentée par une variable de type boolean. Deux littéraux seulement peuvent être employés pour représenter une valeur booléenne : true et false.

Voici une déclaration et une initialisation d'une valeur booléenne:

```
//déclare et initialise en même temps  
boolean isOK = true ;
```

Types primitifs

Types primitifs : texte

Un caractère isolé est représenté à l'aide du type `char` .

Une valeur de type `char` représente un caractère UNICODE (UTF-16 : 16bits, non signé) qui permet de représenter des caractères très divers (langues orientales, symboles).

Un littéral de type **char** doit figurer entre deux apostrophes :

```
'a'           // le caractère a  
'\t', '\n', '\r' // une tabulation, ligne, retour chariot  
'\f', '\b', '\'' // form-feed , backspace , apostrophe, \
```



Une chaîne de caractères peut être représentée au moyen du type `String`. Le type `String` n'est pas un type primitif mais un type prédéfini représenté par une classe.

Types primitifs

Types primitifs : numériques entiers

- Il existe quatre types entiers dans le langage Java : byte, short, int, et long.
- Tous ces types entiers sont des nombres signés.
- La spécification du langage définit une implantation portable indépendante de la plate-forme (ordre des octets, représentation en complément à deux).

Taille implantation	Type	Plage valeurs
8 bits	byte	$-2^7 \dots 2^7 - 1$
16 bits	short	$-2^{15} \dots 2^{15} - 1$
32 bits	int	$-2^{31} \dots 2^{31} - 1$
64 bits	long	$-2^{63} \dots 2^{63} - 1$

Types primitifs

Types primitifs : numériques flottants

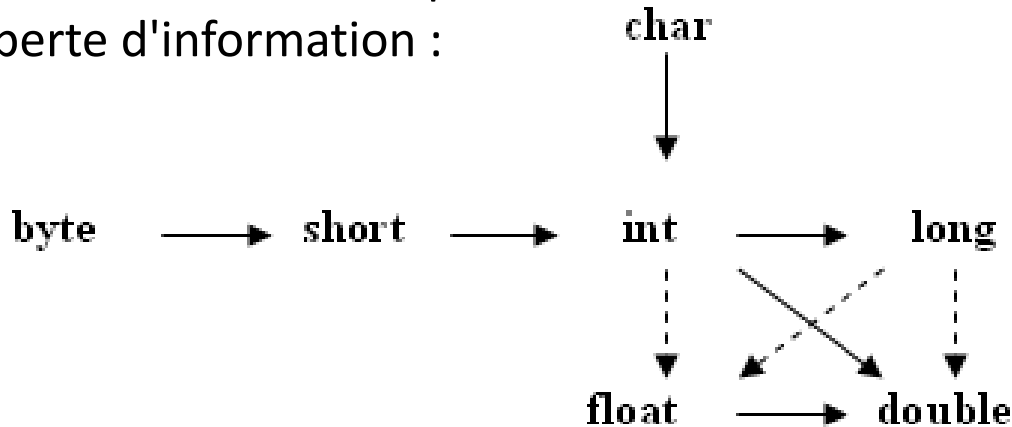
Il existe deux types numériques flottants dans le langage Java : float et double
La spécification du langage définit une implantation portable indépendante de la plate-forme (I.E.E.E 754).

Taille implantation	Type
32 bits	float
64 bits	double

Types primitifs

Types primitifs : numériques (conversion de type)

Les flèches pleines indiquent les conversions automatiques sans perte d'information, les flèches en pointillés, les conversions automatiques avec une possible perte d'information :



- la conversion (forcée) est possible en java, mais vous risquez de perdre des informations

```
double x = 9.997;
int nx = (int) x;
```

- Le type boolean ne peut pas être converti en entier

Types primitifs : Constantes

En java, le mot-clé `final` est utilisé pour définir une constante.

```
public class Constants{  
    public static void main(String[] args)  
    {  
        final double PI= 3.14;  
    }  
}
```

- Le mot-clé `final` indique que vous ne pouvez affecter à la variable une valeur qu'une seule fois, puis sa valeur est fixée une fois pour toutes.

 Il est d'usage de nommer des constantes avec des majuscules.

Types primitifs

Types primitifs : Récapitulatif

- Types arithmétiques

byte	8 bits	signé	(-128 , 127)
short	16 bits	signé	(-32768 , 32767)
int	32 bits	signé	(-2147483648 , 2147483647)
long	64 bits	signé	(-9223372036854775808, 9223372036854775807)
float	32 bits	signé	(1.4E-45 , 3.4028235E38)
double	64 bits	signé	(4.9E-324 , 1.7976931348623157E308)

- Type caractère

char	16 bits	non signé UNICODE2
------	---------	--------------------

- Type booléen

boolean	1 bit	deux valeurs possibles : true ou false
---------	-------	--



Les types de base s'écrivent en minuscules (int, float..)

Exercice d'Application 2

Dans un programme Java, vous devez stocker les informations suivantes :

- le nombre d'étudiants en première année (variable `etudiants`)
- le budget d'un organisme (variable `budget`)
- le nombre π (variable `pi`)
- une mesure de température (variable `temp`)
- la présence d'un étudiant dans un local (variable `present`)
- un caractère tapé sur un clavier (variable `caractere`)

Quelle déclaration de variable choisirez vous pour ces données?

Opérateurs appliqués sur les types primitifs

Les opérateurs arithmétiques

- S'appliquent sur les types **entiers** ou **réels**
 - **+**, **-**, *****, **/**, **%** (modulo), **+=**, **-=**, ***=**, **/=**

```
int x, y, z;  
y = 2;  
z = 3;  
  
x = y * z ;      //x vaut 6  
x += 2 ;         //x vaut 8  
y = x / 4 ;      //y vaut 2  
y =x % 2 ;       //y vaut 0
```

 Les opérateur = et += peuvent être utilisés sur des variables de type « String ». **Donnez-moi un exemple.**

Opérateurs appliqués sur les types primitifs

Opérateurs unaires

- S'appliquent à un seul opérande de type **entier** ou **réel**

-, --, ++

```
int x, y;  
x = 3;  
  
y = -x ;           //y vaut -3  
y = ++x ;          //y vaut 4, x vaut 4  
y = x-- ;          //y vaut 4, x vaut 3
```

- La pré et la post-incrémentation diminuent le nombre de lignes de byte code générées

Opérateurs de comparaison

- S'appliquent sur des **entiers, booléens, réels**
==, !=, <=, >, >=
- Ces opérateurs retournent une valeur du type boolean

```
{  
    boolean droitDeVote;  
    int age;  
  
    droitDeVote = (age >= 18) ;  
}
```


Opérateurs logiques

- S'appliquent au type **boolean**
 - **!** (**not**) , **&&** (**and**) , **||** (**or**)
 - **&**, **|**
- Retournent un type **boolean**
- **&** renvoie « true » si les deux expressions renvoient « true »
- **&&** a le même comportement mais n'évalue pas la seconde expression si la première est « false »
- **|** renvoie « true » si l'une des deux expressions renvoie « true »
- **||** a le même comportement mais n'évalue pas la seconde expression si la première est « true »

Exercice d'Application 4

```
class Test {
    public static void main(String [] args) {
        int x = 0;
        int y = 0;
        while ( x < 5 ) {
            
            System.out.print(x + " " + y + " ");
            x = x + 1;
        }
    }
}
```

Code candidat
↙

(A) Code candidat
`Y = x - Y;`

(B) Code candidat
`Y = Y + x;`

(C) Code candidat
`Y = Y + 2;`
`if( y > 4 ) {`
 `y = y - 1;`
`}`

(D) Code candidat
`x = x + 1;`
`Y = Y + x;`

(E) Code candidat
`if ( y < 5 ) {`
 `x = x + 1;`
 `if ( y < 3 ) {`
 `x = x - 1;`
 `}`
`}`
`Y = Y + 2;`

Sortie

- (1)** 22 46
- (2)** 11 34 59
- (3)** 02 14 26 38
- (4)** 02 14 36 48
- (5)** 00 11 21 32 42
- (6)** 11 21 32 42 53
- (7)** 00 11 23 36 410
- (8)** 02 14 25 36 47

Faites correspondre chaque « code candidat » avec la sortie convenable.

Priorité des opérateurs

- Java a des règles bien définies pour spécifier l'ordre dans lequel les opérateurs d'une expression sont évalués lorsque l'expression a plusieurs opérateurs. Par exemple, la multiplication et la division ont une priorité plus élevée que l'addition et la soustraction. Les règles de priorité peuvent être annulées par des parenthèses explicites.
- **Ordre de priorité**
Lorsque deux opérateurs partagent une opérande, l'opérateur ayant la priorité la plus élevée commence en premier. Par exemple, $1 + 2 * 3$ est traité comme $1 + (2 * 3)$, car la multiplication a une priorité plus élevée que l'addition.
- **Associativité**
Lorsqu'une expression a deux opérateurs avec la même priorité, l'expression est évaluée en fonction de son associativité. Par exemple, $a = b = c = 5$ est traité comme $a = (b = (c = 5))$, les trois variables ont la valeur 5, car l'opérateur $=$ a une associativité de droite à gauche.

Priorité des opérateurs

- Le tableau suivant montre la priorité de quelques opérateurs Java à partir de la priorité la plus élevée à la priorité la plus faible

Niveau	Opérateur	Description	Exemple	Associativité
1	++	incrémentement postfixe	x++	Droite à gauche
	--	décrémentement postfixe	x--	Droite à gauche
2	++	incrémentement préfixe	++x	Droite à gauche
	--	décrémentement préfixe	--x	Droite à gauche
	!	NON logique	! x	Droite à gauche
3	*	multiplication	x * y	Gauche à droite
	/	division	x / y	Gauche à droite
	%	modulo	x % y	Gauche à droite
4	+	addition	x + y	Gauche à droite
	-	soustraction	x - y	Gauche à droite
5	<	plus petit que	x < y	Gauche à droite
	<=	plus petit ou égal à	x <= y	Gauche à droite
	>	plus grand que	x > y	Gauche à droite
	>=	plus grand ou égal à	x >= y	Gauche à droite
6	==	égal	x == y	Gauche à droite
	!=	différent	x != y	Gauche à droite
7	&&	ET logique	x && y	Gauche à droite
8		OU logique inclusif	x y	Gauche à droite
9	? :	opérateur conditionnel	x ? y : z	Droite à gauche
10	=	affectation	x = y	Droite à gauche
	+=, -=, *=, /=, %=	affectations composées	x += y	Droite à gauche

Exercice d'Application 5

```
class Priorite {  
    public static void main(String[] args) {  
        int a = 10, b = 5, c = 1, result;  
        result = a-++c-++b;  
        System.out.println(result); }  
}
```

1. Quelle est la valeur de la variable result?

Exercice d'Application 5

2. Ajoutez des parenthèses aux expressions suivantes pour les rendre plus claires:

$7 + 2 * 3$

$1 - 2 - 3$

$1 + 2 * 3 / 4$

$3 * 5 - 2 + 1 - 8 / 2 * 3$

$-9 >= 6 \ \&\& \ \text{true} \ || \ 7 * 3 < 2$

3. Que donne l'expression suivante:

$2 * 7 + 12 >= 17 \ -1 / 2.0 \ || \ !\text{true}$

Exercice d'Application 5

4. C'est quoi le résultat de ce code:

```
System.out.println("1 + 2 = " + 1 + 2);  
System.out.println("1 + 2 = " + (1 + 2));
```

5. C'est quoi le résultat de ce code:

```
System.out.println(1 + 2 + "abc");  
System.out.println("abc" + 1 + 2);
```